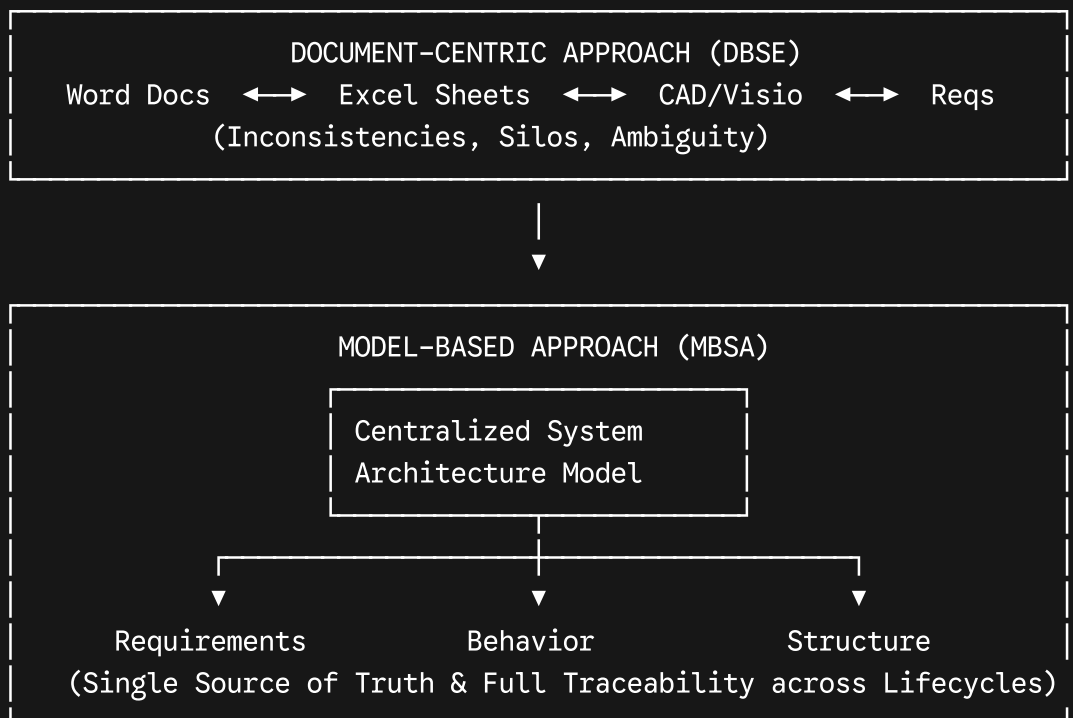


Model-Based System Architecture (MBSA) Design Guide

Executive Summary & Core Concepts

Model-Based System Architecture (MBSA) is a discipline within Model-Based Systems Engineering (MBSE) that puts formalized, digital models at the center of system design. Unlike traditional **Document-Centric Systems Engineering (DBSE)**—where information is spread across static, disconnected specifications, text documents, and spreadsheet trackers—MBSA establishes an integrated, centralized system model as the **Single Source of Truth (SSoT)**.

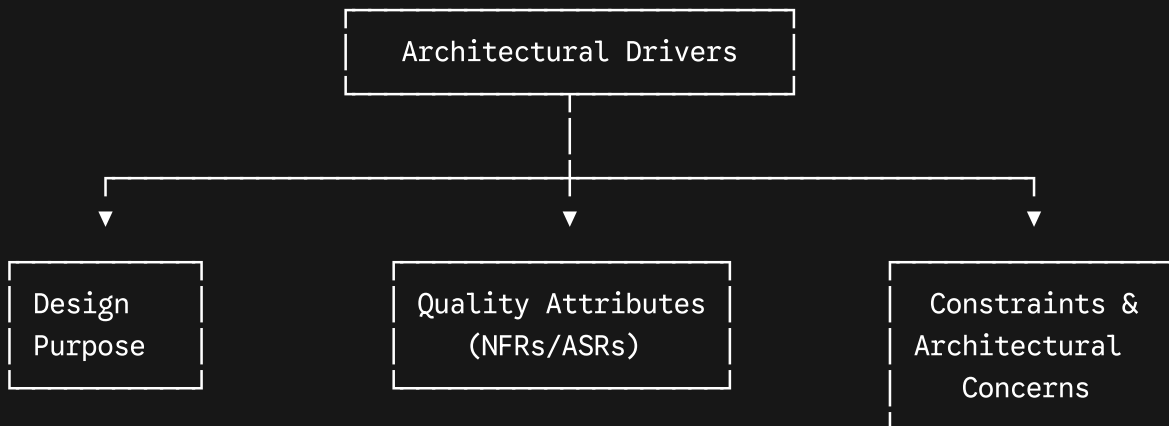


Key Values of MBSA

- **Elimination of Inconsistencies:** Changes in system parameters automatically propagate through connected model elements.
- **Enhanced Communication:** Provides visual representations across multidisciplinary engineering teams (Software, Mechanical, Electrical).
- **Early Analysis & Simulation:** Enables verification and validation (*V&V*) early in the lifecycle before physical prototyping.

Architectural Drivers & Principles

System Architecture is defined as the fundamental concepts or properties of a system in its environment, embodied in its elements, relationships, and design principles.



1. Architectural Drivers

- **Design Purpose:** Clarifies whether design is performed for pre-sales estimation, exploratory prototyping, or full production release.
- **Architecturally Significant Requirements (ASRs):** Critical non-functional requirements (NFRs) such as performance, safety, availability, security, and modifiability.
- **Constraints:** Regulatory standards, budget limits, delivery schedules, legacy technology mandates, and physical limits.

2. Core Architecture Patterns & Principles

- **Separation of Stable from Unstable Parts:** Abstract system functions (stable over decades) from hardware implementations and transient software libraries (rapidly evolving).
- **Cohesion and Coupling:** High cohesion within modules; minimal and clean coupling between modules.
- **Co-existence of Views and Models:** A single integrated model yields multiple specialized diagrams (views) tailored to specific stakeholders.
- **Separation of Definition, Usage, and Run-time:** Distinction between block definitions (type), block instances (usage in context), and dynamic runtime instances.

Architectural Perspectives & Viewpoints (RFLP Framework)

Systems architectures are represented using complementary perspectives. The **RFLP Framework** (Requirements, Functional, Logical, Physical) forms a standard backbone for structuring system perspectives.

RFLP ARCHITECTURE LAYERS	
Requirements	Stakeholder needs, constraints, and test scenarios
Functional	Technology-independent functions & data flows (FAS)
Logical	Abstract component structure & domain concepts
Physical	Concrete hardware, software deployment, & interfaces

Perspective Summary

Perspective	Objective	Typical SysML Diagrams
Requirements Perspective	Captures capabilities, non-functional demands, and constraints.	Requirements Diagram, Use Case Diagram
Functional Perspective	Describes system behavior independent of hardware/software assignments.	Activity Diagram, Sequence Diagram, State Machine
Logical Perspective	Defines high-level component structures and interaction interfaces.	Block Definition Diagram (BDD), Internal Block Diagram (IBD)
Physical Perspective	Specifies exact hardware parts, wiring, software nodes, and physical packaging.	Internal Block Diagram (IBD), Parametric Diagram



Key Methodologies: FAS, SYSMOD, and ADD

1. Functional Architecture for Systems (FAS Method)

Developed by Lamm & Weillkiens, the **FAS Method** converts use case analysis into a clean functional architecture:

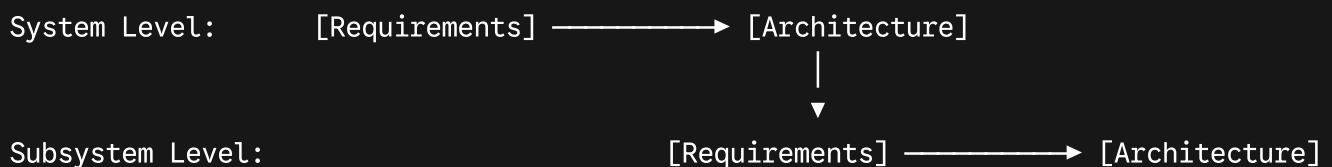
1. **Use Case & Sequence Analysis:** Map system interactions using sequence diagrams.
2. **Function Extraction:** Derive fine-grained actions/activities from execution steps.
3. **Function Grouping:** Group related functions into Functional Groups using heuristics (cohesion, coupling, common information flow).
4. **Functional Block Allocation:** Synthesize functional blocks and define functional interfaces.

Sequence Diagrams → Extract Functions → Apply Heuristics → Functional Architect

2. SYSMOD Zigzag Pattern

The **SYSMOD Zigzag Pattern** illustrates the iterative refinement loop between problem domain (requirements) and solution domain (architecture) across abstraction levels:

System Requirements → System Architecture → Subsystem Requiremen



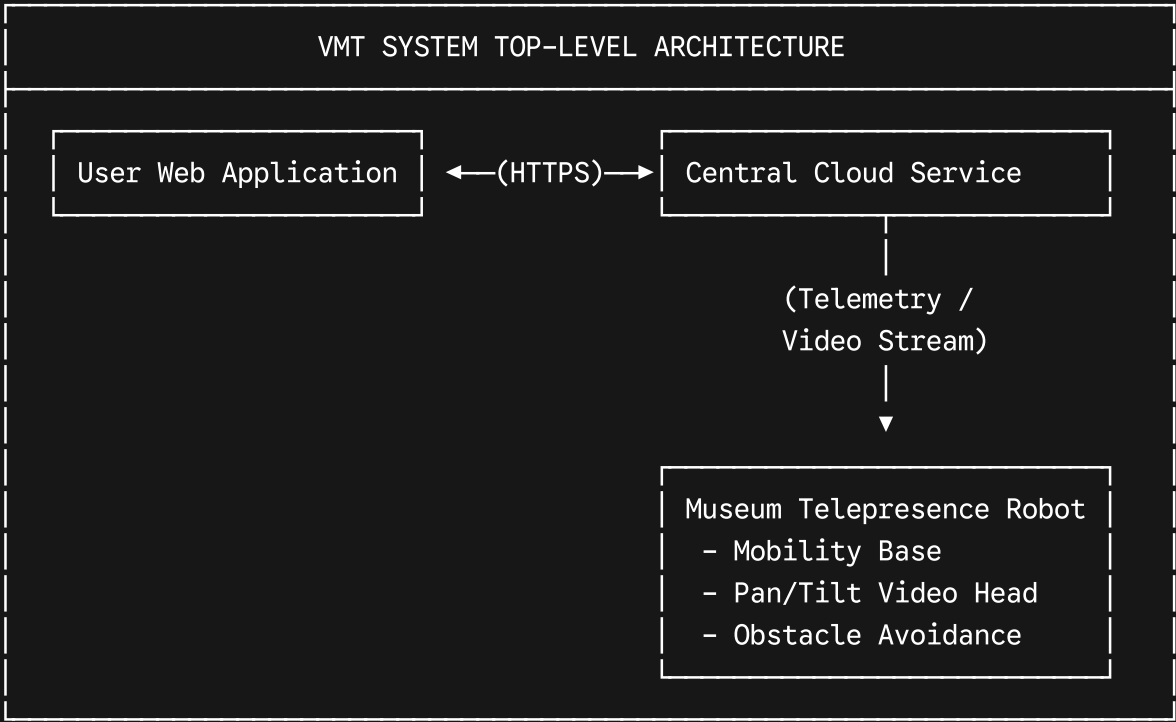
3. Attribute-Driven Design (ADD 3.0)

Focused on software-intensive architectures, **ADD 3.0** systematically satisfies ASRs through iterative steps:

1. Review Inputs (drivers, constraints, quality attributes).
2. Establish iteration goal by selecting primary drivers.
3. Choose elements of the system to refine.
4. Choose design concepts (tactics, architectural patterns, reference architectures).
5. Instantiate elements, allocate responsibilities, and define interfaces.
6. Sketch views and record design rationale.
7. Perform early analysis and review against iteration goal.

Reference Application: Virtual Museum Tour (VMT) System

To illustrate these concepts, consider the **Virtual Museum Tour (VMT) System**, which enables remote users to navigate an automated telepresence robot through museum galleries:

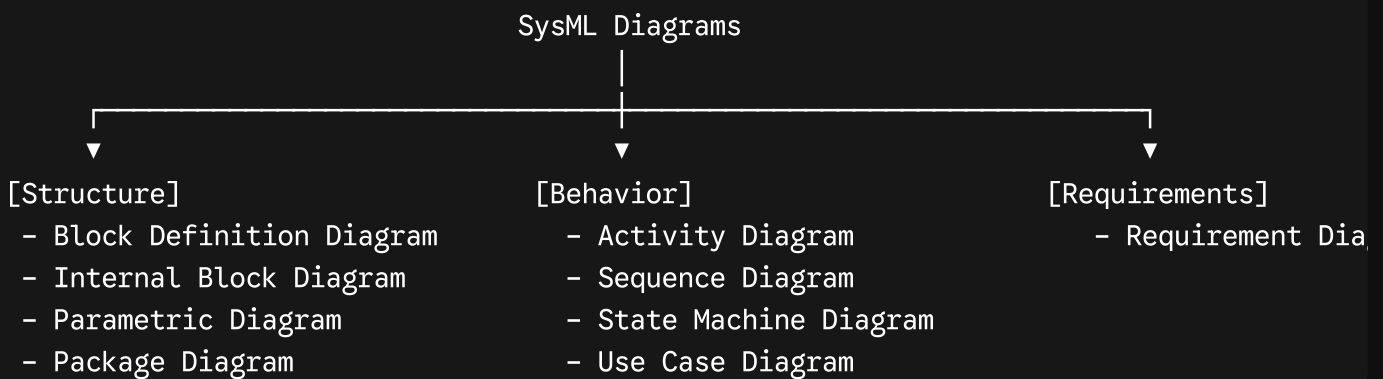


Traceability Mapping Matrix for VMT

Stakeholder Goal	System Requirement	Functional Block	Physical Block
Remote Access	Stream live HD video under 300ms latency	Video Processing & Compression	Hardware H.265 Encoder + Wi-Fi Module
Artwork Safety	Prevent collisions with exhibits at all times	Obstacle Detection & Braking	LiDAR Array Ultrasonic Proximity Sensors
Interactive Tour	Allow pan/tilt/zoom adjustments by active controller	Camera Motion Control	Pan-Tilt Servo Motor Optical Zoom Lens

Standard Notation: Systems Modeling Language (SysML)

SysML extends UML with 9 primary diagram types organized into **Structure**, **Behavior**, and **Requirements**:



- **Block Definition Diagram (BDD):** Defines system blocks, properties, and composition hierarchies.
- **Internal Block Diagram (IBD):** Internal flow of material, energy, and data between flow ports.
- **Parametric Diagram:** Defines constraints and mathematical equations (e.g., mass budgets, power limits) integrated into the model.

Soft Skills & Organizational Integration

System Architecture is as much a communication discipline as a technical one.

Key Soft Skills for System Architects

- **Stakeholder Negotiation:** Balancing contradictory goals across business managers, domain specialists, and end users.
- **Simplification:** Explaining complex architectural tradeoffs in clear visual forms.
- **Leadership & Coaching:** Mentoring development teams and maintaining technical integrity across iterations.

Organizational Governance

1. **System Architect Role:** Serves as technical lead, coordinating with Project Managers, Systems Engineers, and Domain Experts.
2. **Architecture Frameworks (UPDM / UAF):** Essential for System of Systems (SoS) integration in large-scale defense, aerospace, or enterprise applications.